



SOLUCIONES INFORMÁTICAS PARA LA EMPRESA

TAREA 1

Diseño e implementación de un canal digital de venta instrumentado

UCAM Stereo

E-commerce de audio y tocadiscos vintage

AUTORES

Dusjan Jevtic Santander
Germán Navarro Marín
Javier Mínguez García
Pedro Javier Moreno Gázquez
Víctor Matías García

PROFESOR

Antonio Serrano Fernández

TITULACIÓN

Grado en Ingeniería Informática
Modalidad presencial

Curso 2026/2027

Aplicación: <https://www.ucamstereo.onl>

Repositorio: <https://github.com/DusjanJS/PaginaVenta>

Prototipo académico sin actividad comercial real. Los pagos se realizan exclusivamente en PayPal Sandbox con cuentas ficticias; no se incluyen credenciales personales ni secretos del despliegue.

1 Caso de empresa y arquitectura

UCAM Stereo es una tienda ficticia especializada en audio y escucha analógica. Su catálogo contiene diez productos realistas de cuatro categorías: vinilos, tocadiscos, altavoces y auriculares. La finalidad académica es implementar un front-office que complete una compra y produzca datos maestros, transacciones y eventos reutilizables por otros sistemas.

Flujo y reglas de negocio

- Portada, catálogo por categorías, búsqueda, marca, precio y ordenación.
- Ficha con descripción, especificaciones, variante y stock disponible.
- Carrito, checkout con datos de envío, resumen y pedido único.
- IVA incluido del 21 %, envío de 6,90 € y envío gratuito desde 300 € tras descuentos.
- Descuento de bienvenida del 10 % durante el primer mes para clientes registrados.
- Compra registrada o como invitado y pago de prueba en

PayPal Sandbox.

- Estados: creado, pagado, pendiente de preparación, enviado, entregado, cancelado o con incidencia.
- Back-office para pedidos, estados, incidencias y eventos.

Separación de responsabilidades

El navegador presenta la interfaz y nunca decide el importe definitivo. La API autentica, autoriza, valida entradas, recalcula precios y coordina pedidos y pagos. PostgreSQL conserva usuarios, sesiones, variantes, stock, pedidos, pagos, incidencias y eventos. Esta separación evita que el cliente acceda directamente a la base o modifique totales.

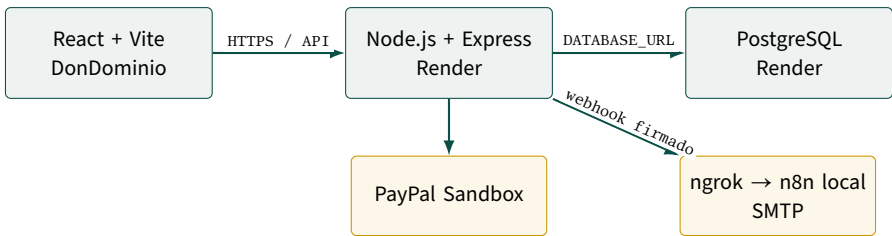


Figura 1. Arquitectura desplegada. ngrok solo expone el webhook local de n8n al backend.

Ámbito	Elección	Justificación
Interfaz	React 19, Vite y CSS	Componentes reutilizables y compilación estática compatible con DonDominio.
Servidor y datos	Express, Node.js y PostgreSQL	Reglas en servidor, transacciones, claves foráneas y bloqueo de stock.
Despliegue	DonDominio + Render	Hosting estático para el frontend y servicio web con PostgreSQL para la API.
Pago y automatización	PayPal Sandbox + n8n/ngrok	Pago realista sin cargos y correo automático desde un flujo local.

2 Implementación del canal digital

Frontend

La aplicación React usa rutas para inicio, catálogo, producto, carrito, checkout, confirmación y seguimiento de pedido, acceso, soporte, administración y páginas informativas. `ProductsContext` obtiene el catálogo desde la API; `CartContext` mantiene las líneas en `sessionStorage`; y `AuthContext` gestiona registro, sesión, cuenta y pedidos. Cinco productos tienen alternativas de acabado y otros cinco alternativas de color; cada variante posee SKU, imagen y stock propios.

El checkout solicita nombre, correo, teléfono, dirección, ciudad, provincia, código postal y país. Antes del pago muestra líneas, variantes, descuento, envío, base imponible, IVA y total. El stock visible y las cantidades del carrito proceden del backend. La interfaz muestra permanentemente que es un prototipo académico y que PayPal opera en Sandbox.

Backend, seguridad y persistencia

La API REST se organiza por autenticación, productos, pedidos, PayPal, eventos e incidencias. Al crear un pedido recibe identificadores y cantidades, consulta el catálogo y stock y recalcula todos los importes con las reglas compartidas. Durante la captura de PayPal comprueba orden, moneda, importe y estado; una transacción bloquea las variantes con `FOR UPDATE`, marca el pedido pagado y descuenta el stock sin duplicar capturas.

Las contraseñas se almacenan mediante `bcrypt` con sal aleatoria; las sesiones conservan únicamente el hash SHA-256 del token. El servidor revisa el rol en cada ruta administrativa. Los secretos de PostgreSQL, PayPal, n8n, seguimiento y CORS se configuran como variables de entorno y los archivos `.env` reales están excluidos del repositorio.

Despliegue e integraciones

Elemento	Funcionamiento desplegado
Frontend	Archivos estáticos en DonDominio, accesibles en https://www.ucamstereo.on1 . Se compilan con <code>VITE_API_URL=https://ucam-stereo-api.onrender.com/api</code> .
API y base	Servicio web y PostgreSQL en Render. <code>render.yaml</code> vincula <code>DATABASE_URL</code> ; <code>init-db.js</code> inicializa solo una base vacía.
PayPal	El backend crea y captura órdenes en Sandbox. <code>PayPal-Request-Id</code> y la referencia guardada evitan operaciones duplicadas.
Correo y rastreo	Tras el pago, el backend crea un enlace firmado de seguimiento y envía los datos a n8n. ngrok publica temporalmente el webhook del n8n local; SMTP entrega el correo al comprador.

Evidencias funcionales

El proyecto incorpora datos de prueba, validación HTML y de servidor, manejo de errores y control de acceso. Existen cuentas ficticias de cliente y administración documentadas en el README. El comprador puede consultar sus pedidos desde la cuenta o mediante el enlace firmado recibido por correo; un invitado no obtiene acceso a pedidos ajenos.

3 Modelo de datos

El esquema PostgreSQL v5 contiene catorce tablas. Separa datos maestros, seguridad y operaciones; el diagrama completo aparece en la página 8.

Entidades	Tipo	Finalidad
categoría, producto, variante_producto	Maestro	Catálogo, contenido de ficha, SKU, imagen, estado y stock por variante.
conexión, producto_conexión	Maestro	Prestaciones de conectividad y relación N:M con productos.
cliente, sesión	Identidad y seguridad	Usuarios registrados, invitados, administradores y sesiones con token hash.
carrito, línea_carrito	Transaccional	Modelo previsto para carritos persistentes; el carrito operativo actual se conserva en el navegador.
pedido, línea_pedido	Transaccional	Cabecera, envío, importes, estado y copia histórica de cada artículo comprado.
pago	Transaccional	Identificadores y resultado del pago realizado en PayPal Sandbox.
incidencia	Transaccional	Solicitudes de soporte con pedido opcional.
evento	Trazabilidad	Tipo, fecha, referencias, sesión web y datos adicionales JSONB.

Tabla 1. Entidades principales y responsabilidad.

Relaciones e integridad

- Categoría–producto y producto–variante: relaciones 1:N.
- Producto–conexión: N:M mediante producto_conexion.
- Cliente–pedido, pedido–línea y pedido–pago: 1:N.
- Evento puede referenciar opcionalmente cliente, producto y pedido.
- Claves foráneas y una clave compuesta impiden usar una variante de otro producto.
- Restricciones CHECK validan cantidades, stock, importes, roles, estados y descuentos.
- Índices parciales garantizan una variante base, un carrito activo y un pago aprobado cuando corresponde.
- Triggers mantienen fechas de actualización; una secuencia genera UC-AAAA-NNNNN.

Consistencia de la compra

Las líneas guardan nombre, marca, variante y precio para conservar el historial aunque cambie el catálogo. El servidor calcula subtotal, descuento, envío, IVA y total; PostgreSQL aplica restricciones equivalentes. La creación del pedido y la captura se ejecutan dentro de transacciones. El stock se bloquea antes de descontarlo, evitando que dos compras simultáneas consuman las mismas últimas unidades.

Datos de prueba

Datos.sql carga cuatro categorías, diez productos, veinte variantes, conexiones, cuentas ficticias y ejemplos de pedidos, pagos, incidencias y eventos. Consultas.sql permite revisar catálogo, stock, clientes, pedidos, pagos, incidencias, eventos y coherencia. Los scripts completos se entregan como archivos, sin reproducirlos en esta memoria.

4 Instrumentación de eventos

Los eventos se guardan en la tabla evento de PostgreSQL. El frontend llama a `POST /api/eventos` para acciones de navegación y compra; el backend valida el tipo, completa las referencias disponibles y registra desde el servidor los hechos que dependen de una operación confirmada.

Evento	Momento y datos principales
<code>product.viewed</code>	Apertura de una ficha; producto, SKU y categoría.
<code>cart.item_added</code>	Incorporación al carrito; producto, variante, cantidad y precio.
<code>checkout.started</code>	Entrada al checkout con líneas en el carrito.
<code>order.created</code>	Pedido confirmado por el backend; número, total y líneas.
<code>payment.simulated</code>	Resultado de la captura en PayPal Sandbox.
<code>support.requested</code>	Incidencia guardada por el servidor, con pedido opcional.
<code>order.status_changed</code>	Cambio administrativo; estado anterior y nuevo.

Tabla 2. Seis eventos mínimos del enunciado y un evento adicional de gestión.

Generación, almacenamiento y consumo

Cada fila conserva tipo, fecha, cliente, producto o pedido cuando existen, identificador de sesión web y un payload JSONB. Se almacena únicamente la información necesaria para la trazabilidad. La administración permite consultar los eventos y el backend puede exportarlos como JSON o CSV. Consultas.sql incluye agregaciones y verificaciones.

En la Tarea 2 estos datos pueden alimentar un cuadro de mando o un proceso de automatización. Permiten calcular productos vistos, productos añadidos, inicios de checkout, pedidos, pagos, solicitudes de soporte, conversión y abandono. El número de pedido y las marcas temporales facilitan relacionar el recorrido del cliente sin incorporar secretos ni datos de pago.

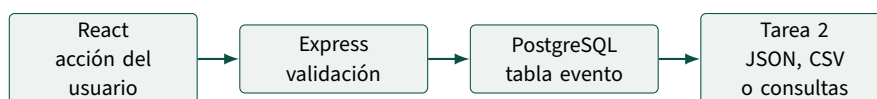


Figura 2. Ciclo de generación y reutilización de eventos.

Trazabilidad

Los cambios de estado quedan ligados al pedido y a la cuenta administrativa. El pago guarda la orden y captura de PayPal; el correo contiene una URL firmada, no un identificador abierto. Si el envío a n8n falla, el pedido y el pago permanecen almacenados y el fallo no revierte la compra.

5 Limitaciones, riesgos y validación

Limitaciones conocidas

Área	Limitación y consecuencia
Render gratuito	El backend puede suspenderse sin tráfico y la primera petición tarda. La base gratuita tiene duración limitada y no aporta copias administradas.
Dominio	La dirección validada es <code>https://www.ucamstereo.onl</code> ; el dominio raíz presentó un problema de certificado o redirección.
n8n y ngrok	El recibo requiere portátil encendido, n8n activo y túnel vigente. Su fallo no elimina el pedido, pero puede impedir el correo.
Cuentas	No hay verificación de correo, recuperación de contraseña ni segundo factor. Las cuentas <code>@ucam.test</code> son públicas y solo sirven para evaluación.
Eventos e imágenes	Un cliente desconectado puede perder un evento; las licencias de imágenes deberían revisarse antes de un uso comercial.

Validación realizada

- Los scripts se ejecutaron en PostgreSQL 16 local y el esquema se inicializó en Render PostgreSQL; las consultas no detectaron pedidos, pagos o stocks incoherentes.
- El 7 de octubre de 2026, las diez pruebas automáticas del backend finalizaron correctamente y `npm run build` generó el frontend de producción sin errores.
- Se comprobaron el dominio, `/health`, el catálogo de la API, las rutas SPA y el control de acceso de cliente, invitado y administrador.
- Una compra en PayPal Sandbox terminó en `COMPLETED`, descontó stock y generó pedido. Con n8n y ngrok activos se recibió el correo y funcionó el enlace firmado de seguimiento.

Seguridad y privacidad

Solo se usan cuentas y datos ficticios. El repositorio contiene ejemplos de variables, nunca los secretos reales. El navegador no recibe `DATABASE_URL`, credenciales SMTP, secreto de PayPal ni claves de firma. Las rutas validan entradas y permisos en el servidor; los pagos y saldos de Sandbox no representan dinero real.

Conclusión

El prototipo completa el flujo exigido: catálogo, variantes y stock, carrito, checkout, reglas comerciales, pago de prueba, pedido único, estados, administración, soporte y eventos persistentes. La arquitectura separa interfaz, negocio y datos, y deja evidencias reutilizables para la siguiente tarea. Las mejoras prioritarias son alojar n8n de forma permanente, añadir copias de seguridad y recuperación de cuenta y unificar por completo el catálogo en PostgreSQL.

6 Punto de partida y uso de IA generativa

Declaración del punto de partida

UCAM Stereo se desarrolló específicamente para esta tarea. No se partió de un comercio electrónico completo ni de una plantilla comercial lista para producción. La interfaz se implementó con React, Vite, JSX y CSS propio a partir de una referencia visual creada con Figma Make; después se adaptó a los componentes, contextos y servicios reales. La API, el modelo PostgreSQL y las reglas de negocio se construyeron para el proyecto. Supabase se utilizó durante comprobaciones iniciales, pero el despliegue final usa Render para la API y PostgreSQL, por lo que Supabase no interviene en la web actual.

Declaración de herramientas y partes asistidas

ChatGPT y Codex (OpenAI) se utilizaron para analizar requisitos, revisar y modificar frontend y backend, diagnosticar PayPal, n8n, ngrok, Git, Render y DonDominio, y consolidar la documentación. **Claude (Anthropic)** ayudó con el modelo relacional, scripts SQL y estructura inicial de la memoria. **Figma Make** produjo una referencia visual que el grupo trasladó y adaptó al frontend.

Las partes relevantes asistidas fueron componentes, contextos, estilos y formularios del frontend; autenticación, pedidos, PayPal y webhook del backend; esquema, datos y consultas de PostgreSQL; configuración del despliegue y documentación. El uso no se limitó a consultas: se generaron o modificaron fragmentos de código y texto que después se inspeccionaron y probaron.

Errores detectados y cambios del grupo

- Se corrigió la función de ngrok: expone el webhook local de n8n al backend de Render, no conecta el frontend con la API.
- Se rechazaron instrucciones que suponían código inexistente en Checkout.jsx; antes de editar se revisó el archivo real.
- Se eliminaron afirmaciones que mantenían Supabase como producción después de migrar a Render PostgreSQL.
- Se cambió la unicidad del correo para que un invitado pueda comprar varias veces sin impedir usuarios registrados duplicados.
- Se revisaron orden, captura, moneda, importe e idempotencia de PayPal y se aclaró que los saldos de Sandbox son ficticios.

Validación y responsabilidad

El grupo aceptó propuestas solo después de contrastarlas con el repositorio, compilar, ejecutar pruebas o recorrer manualmente la función afectada. Los cambios finales, la integración, las variables del despliegue y la responsabilidad sobre lo entregado corresponden al grupo. Todos los miembros deben comprender y poder defender las partes esenciales del sistema.

Miembro	Responsabilidad principal
Dusjan Jevtic Santander	Frontend y repositorio.
Germán Navarro Marín	Dominio, hosting y apoyo al despliegue.
Javier Mínguez García	Backend.
Pedro Javier Moreno Gázquez	Base de datos y datos de prueba.
Víctor Matías García	Modelo de datos y diagrama entidad-relación.

7 Esquema de entidades

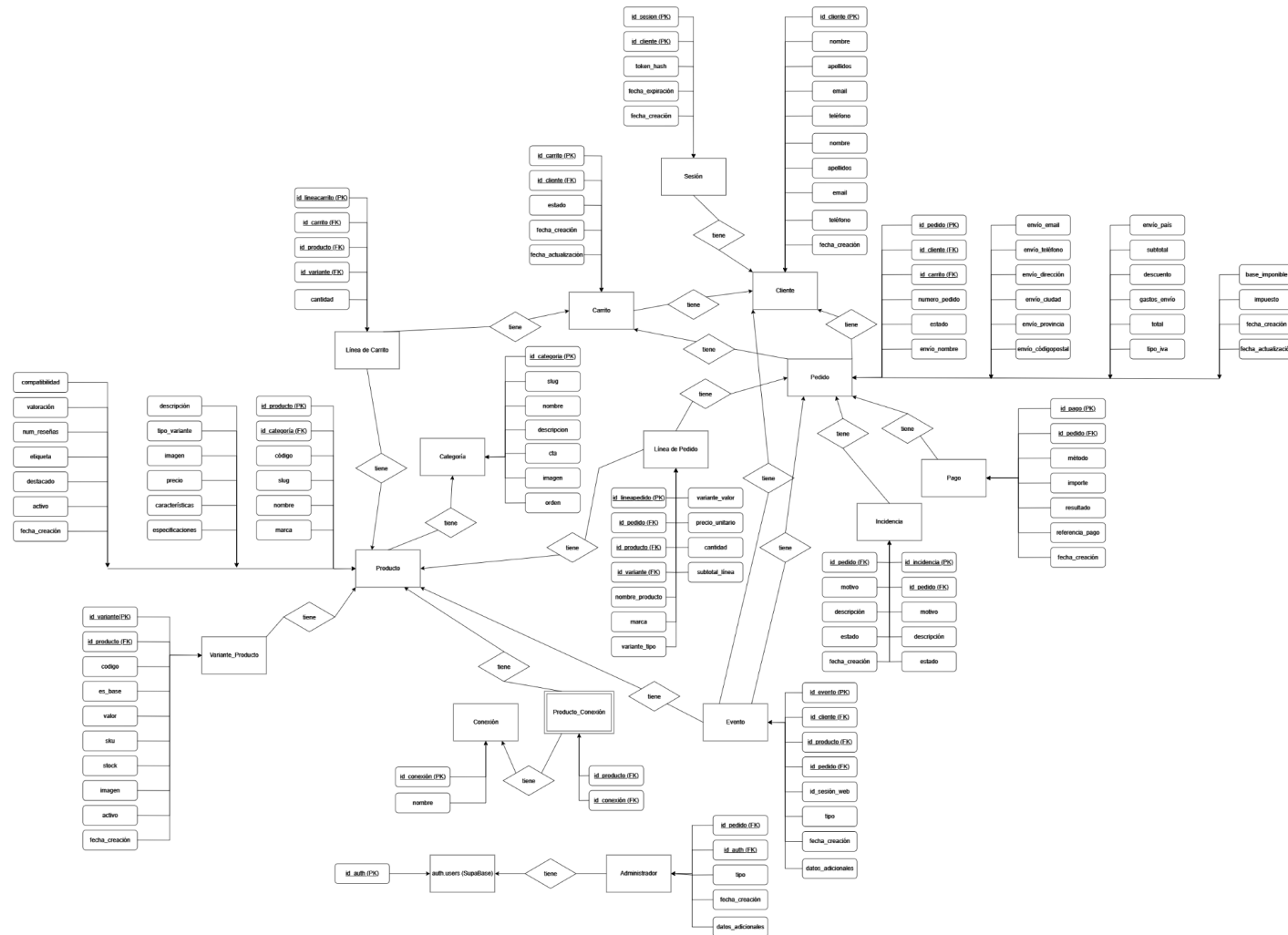


Figura 3. Diagrama entidad-relación de UCAM Stereo.

El esquema implementado se entrega en docs/Estructura.sql; los datos ficticios y las consultas de verificación se incluyen en docs/Datos.sql y docs/Consultas.sql. El diagrama editable se aporta en formato draw.io.